

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/370123416>

Design & Verification of AMBA AHB-Lite Memory Controller

Conference Paper · March 2023

DOI: 10.1109/CCWC57344.2023.10099257

CITATIONS

8

READS

964

4 authors, including:



[Shrikant Jadhav](#)

San Jose State University

38 PUBLICATIONS 116 CITATIONS

SEE PROFILE

Design & Verification of AMBA AHB-Lite Memory Controller

Anila Kommineni
Dept. of Electrical Engineering
San Jose State University
San Jose, USA
anila.kommineni@sjsu.edu

Manoj Kumar Gundu
Dept. of Electrical Engineering
San Jose State University
San Jose, USA
manojkumar.gundu@sjsu.edu

Youngsoo Kim
Dept. of Electrical and Computer Engineering
Bradley University
Peoria, USA
ykim@bradley.edu

Shrikant Jadhav
Dept. of Electrical Engineering
San Jose State University
San Jose, USA
shrikant.jadhav@sjsu.edu

Abstract—

As technology advances, the on-chip communication bus architecture becomes increasingly prominent in interconnecting various components within the System-on-Chip (SoC). The standard ARM AMBA on-chip interconnect bus is designed as an SoC system's high-performance backbone bus, which supports faster communication with internal and external memories. This paper presented a memory controller design with an AMBA 3 AHB_Lite standard based on a single master and multiple slave model. We verified the design as per the specifications of ARM using a System Verilog verification environment and functional coverage. Various testbench verification environment components such as transaction and generator (which generates the input stimulus), Driver (which drives input data to the Design Under Test (DUT)), Monitor (which monitors the signals from the DUT), and the Scoreboard (which reports about the design working condition) are developed to test single burst, wrapping, and increment bursts of various sizes (4, 8, and 16 beats) with waited transfer responses of the AHB_Lite protocol. We also observed different corner cases during burst and wrap transfer.

Index Terms—Verilog, SoC, Memory Controller, AMBA 3, AHB-Lite, System Verilog, verification.

I. INTRODUCTION

The increasing complexity and size of computer-aided design has made functional verification a huge challenge. Several new verification technologies have emerged in the last decade, and some have established themselves as essential components of the verification process. Functional design verification consumes 40% to 50% of project resources. The design verification cost component accounts for roughly 40% of the total outline cost. A critical role for functional verification is to recognize nonattendance of progress so that bugs can be identified and corrected before being delivered to the customer. If the designer makes a mistake while designing or coding, the Chip will have a bug. If this bug is exploited, the framework will deliver incorrect results in certain circumstances, resulting in chip failure.

Applications like video and image processing require correlation and synchronization between various data sets to produce optimal results. A memory controller is required to perform this task and store the data in memory, which is essential for accelerating the process. Also, a high-speed memory controller is required to improve the latency when employing memory modules in FPGA applications. As per AMBA3 standard, AHB Lite is defined as a less complex bus than AHB since it has a single master. AHB-Lite interface memory controller architecture will be utilized for various AHB transactions to implement reading and writing operations. A processor must interact with multiple other processors, internal memories, and multiple devices to complete a task. This is accomplished using the bus system. AMBA is a well-known open specification for managing the functional blocks that comprise SOC's.

ARM's Advanced Microcontroller Bus Architecture (AMBA) bus protocols are interconnect standards that standardize on-chip communication between different IPs or functional blocks for creating high-performance SOC designs. AMBA includes a variety of protocols based on system requirements, such as Advanced Peripheral Bus (APB), Advanced System Bus (ASB), Advanced High Performance (AHB), or Advanced eXtensible Interface (AXI). The AHB connects components that require more bandwidth on a shared bus. This paper describes the AHB_Lite memory controller and other peripherals. This AHB-Lite system consists of a single master and two slaves, an address decoder (selects the slave), and a slave-to-master multiplexer (used when the slave drive reads data to the master). With the help of these decoders and mux, we can select a slave for each transfer. The memory depth of each slave is 256B, which holds the data, and the memory controller controls the read and writes operations based on slave selection with address and control buses.

We need an advanced verification environment like System Verilog to deliver an error-free design. The most sig-

979-8-3503-3286-5/23/\$31.00 ©2023 IEEE

nificant advantages of using this language are its random constrained-based verification capability and reusability. Random constraint-based verification is useful for verifying large complex designs because it can generate many random test cases and hit corner cases faster. Verification engineers do not always have to create a new verification environment; they can also modify an existing one. These functions are easily accessible in System Verilog. Synopsys VCS is an integrated circuit design and verification software system used to simulate and verify our design and calculate functional coverage.

II. RELATED WORK

Recent advancements in SoC technology have enabled multiple devices to be embedded on a single chip, allowing for compact devices. In order to achieve effective communication, bus-based protocols are used to interconnect the different components. The AHB protocol is the one that is most frequently used for communication. The AHB protocol supports high-speed device communication. AHB is a memory controller that transfers data between the processor and memory. Since memories are so large, processors are gradually getting faster while they are getting slower. SDRAM's latency is not limited to a single cycle, so AHB is idle during that time. As a result, the processor uses bus resources inefficiently to communicate with memory [1].

S. L. Reddy and colleagues developed a memory controller explicitly made for SDRAM. This controller was designed to support the protocol's architecture, which is helpful for high-performance and high-bandwidth devices [2]. New features have been incorporated into the design of the memory controller to improve its overall efficacy. For instance, the controller's internal memory has searched first rather than the memory when a query about recently used data is made.

The paper proposed by Vijayalakshmi et al. aims to improve memory and storage, including FIFO, while also speeding up device access. As a result, the memory controller is designed so that access times should be reduced. MC's job is to manage memory. MC is typically produced on a chip. This paper focuses on creating an AHB memory controller that can also function as AHB slave peripherals [3].

In this paper, Varsha Vishwarkama employs a high-performance architecture that uses a cutting-edge microcontroller bus. One approach to performance enhancement is the "AHB bus." However, because it only allows one master to converse with one slave, it cannot achieve perfect parallelism. AMBA-AHB supports SoC communication standards to enable parallelism and develop high-performance embedded systems [4]. This paper validates the WAITED transfers of AMBA AHB LITE protocol results by creating a reusable verification environment such as a scoreboard, driver, and monitor [5].

The implementation of high-performance embedded SOC platforms is suggested in this paper. The enhanced high-speed AHB protocol with a priority encoder is suggested with an arbitrator and a generic bridge, which improves performance

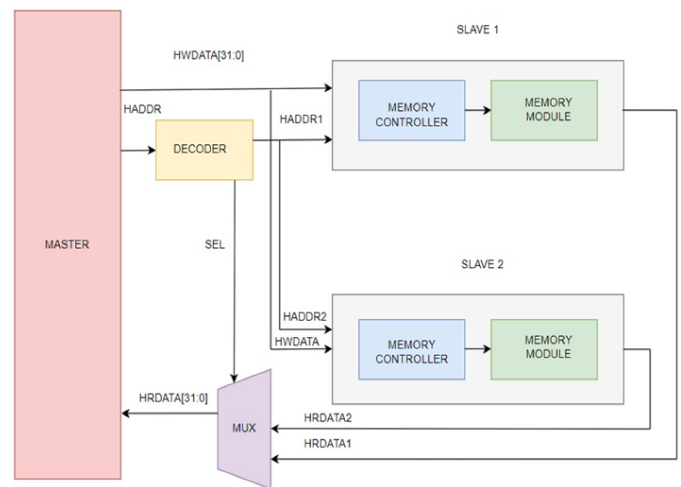


Fig. 1: AHB-Lite Block Diagram

and throughput [6]. This study uses a Xilinx simulator to design and test an efficient AMBA controller for data transaction operations. Some examples are used to illustrate the process of read-and-write operations using the AMBA protocol [7]. This paper suggests a different protocol based on AHB Lite for low-energy applications that will allow direct interactions between masters and slaves connected to the peripheral and system domains [8].

The previous work mentioned the advancement in the memory controller using the AHB-Lite interface and how verification environments help test complex systems. In our work we used verification methodologies like WAITED transfers and developed the verification environment in System Verilog using AMBA AHB-Lite standards.

III. HARDWARE IMPLEMENTATION OF AHB-LITE MEMORY CONTROLLER

Figure 1 represents the AHB Lite memory controller block diagram. It consists of four components, i.e., master, slave, decoder, and multiplexer. It consists of one master and two slave memory models, which interface with the AHB_Lite memory controller. The memory controller controls the READ and WRITE operations of different AHB_Lite transactions. The memory controller receives the read request from the AHB bus (i.e., from the master) and performs the read operation from the starting address location, depending on how the data is to be retrieved, and sends the data from the slave to the AHB bus (i.e., to the master) through the multiplexer. Similarly, the memory controller receive write data (i.e., from the master) and decodes it, and sends it to the corresponding slave device. Some of the decoded data include (1) the size of the burst (4, 8, 16), (2) starting address of the read data, (3) the write data enabled (currently active data), (4) wrapping style (wrap 4, wrap 8, and wrap 16) or incremented style (5) information to write or read from memory.

Figure 2 represents the master interface. The master gives the slaves information to read and write to the memory mod-

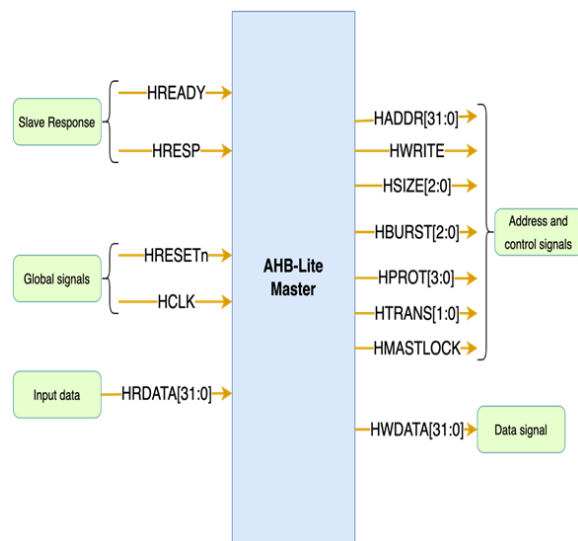


Fig. 2: Master Interface

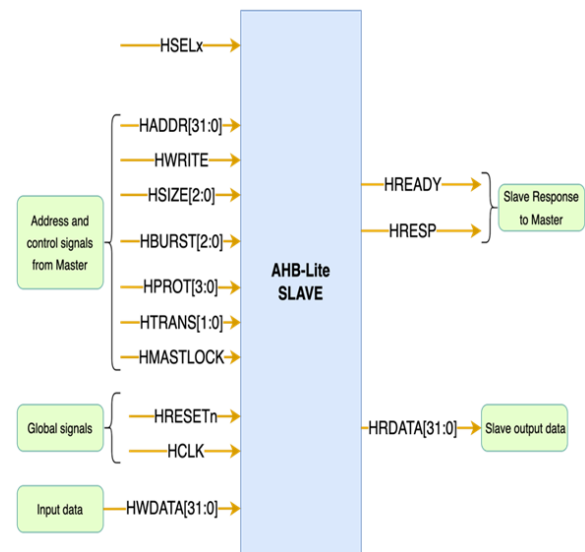


Fig. 3: Slave Interface

ule. A selection signal, HSEL, selects each slave and drives the signal to the corresponding slave. The master also receives two slave response signals, HRESP and HREADY, which indicate whether a transfer was successful or unsuccessful (HRESP) and whether the data phase should be extended (HREADY). The finite state machine of the AHB master consists of idle, address phase, data phase, and wait states. In an idle state, the master sends a request signal to a slave, indicating idle transfer type. The address phase state is used to provide address and control information. In the data phase state, the master sends the HWDATA [31:0] on the data bus whenever the signal HWRITE=1. Whenever HWRITE=0, the master receives HRDATA [31:0]. Finally, in a wait state, the master waits for the slave's response signal, i.e., HREADY. If HREADY='1', the current transaction has been completed and has moved on to the address phase. If HREADY='0', the transaction has not been completed, and the master will remain in the wait state.

The slave interface is shown in Figure 3. AHB-Lite slave recognizes the received address range and responds to a master for a read or write transfer request. Depending on the status of the transaction (completed or failed), the slave sends HREADY='1' and HRESP, i.e., OKAY or ERROR or SPILT to the master; otherwise, the slave sends HREADY='0' if the slave is busy. The finite state machine of the AHB slave consists of idle, address phase, and data phase states. It uses select signal HSELx from the decoder and completes the input transaction, i.e., read or write, depending on the HWRITE value. In an idle state, the slave waits for the master's request, i.e., read or write, and when the corresponding HSELx='1', then the slave goes to the address phase and sends an OKAY or ERROR response, which indicates the successful or unsuccessful transaction of the previous data request. In the address phase state, the slave uses this state to respond back to the master's request. When considering locked transfer and split

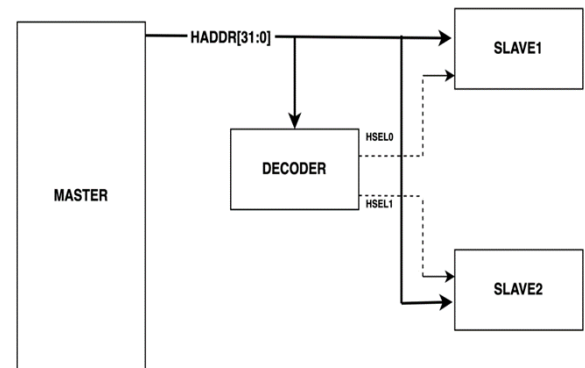


Fig. 4: AHB Decoder

transfer, this state is used to provide address and response information. In the data phase state, the slave responds to the master's request and broadcasts the required data onto the data bus (i.e., read-data bus) (HRDATA[32]).

One hot technique is used to implement a decoder for the AHB implementation, as shown in Figure 4. The centralized command decoder is used in the AHB implementations. The AHB-Lite decoder fetches the address HADDR from the master. It decodes the HADDR to identify the slave selected for performing the transaction. Each AHB-Lite slave has its own slave select signal HSELx. This signal indicates the current transfer intended for the selected slave. When the slave is initially selected, it also monitors the status of HREADY to ensure that the previous bus transfer has been completed before it responds to the current transfer.

The connection of the multiplexer between the master and slave is shown in Figure 5. The decoder controls the multiplexer. Multiplexer, embedded between the master and slave, provides the purpose of multiplexing the fetched data from the slave (i.e., response) back to the master. When there are at least two slaves in AHB-Lite protocol usage, multiplexing is

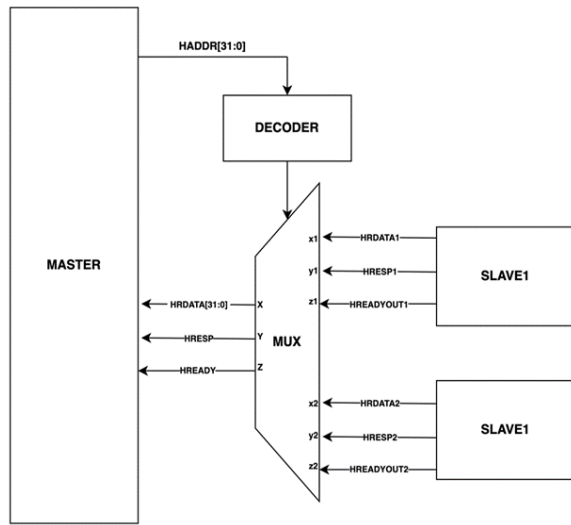


Fig. 5: AHB Multiplexer

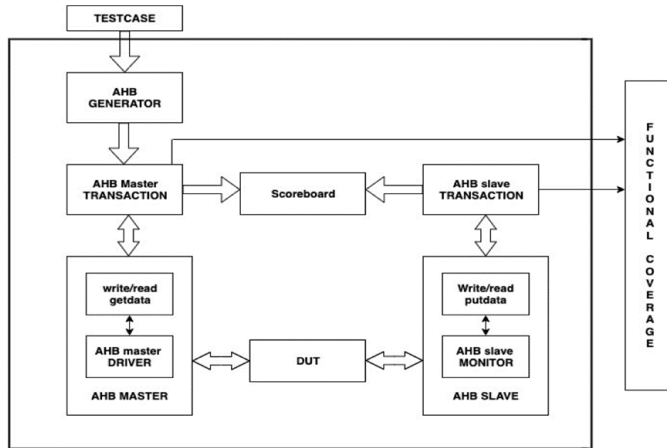


Fig. 6: Verification Block Diagram

mandatory to understand which slave is routing back the data fetched.

IV. SV-BASED VERIFICATION OF AHB-LITE MEMORY CONTROLLER

Figure 6 shows the System Verilog-based verification environment block diagram. System Verilog is a Verilog-based Hardware Description Language (HDL) and Hardware Verification Language (HVL). While it has few design features, the purpose of the language is to verify electronic designs. A stimulus is driven in this DUT based on the test plan and checked DUT output. An object-oriented testbench with different modules and classes is implemented in System Verilog. The different blocks of the verification environment are explained in the following paragraphs.

The *AHB Generator* generates random stimulus using the class transaction and constraints written in the transaction class. The *AHB Driver* component drives the input data to the DUT. The *Driver* calls the get method using the mailbox,

and data is routed from the generator to the *Driver* through the mailbox. The *Driver* pushes this stimulus to the design using the help of a virtual interface. The *Interface* serves as a medium between the *Driver* and the design. All AHB-Lite protocol signals are declared in the *Interface* module. The modports and clocking blocks are used in the *Interface*. The response from the design is collected in the *AHB Monitor*. The *Monitor* track the values, and similar to the *Driver*, the *Monitor* gets the values through the virtual interface. Then these values are directed to the *Scoreboard* using a mailbox. The reference model is created in the *Scoreboard*. The *Monitor* routes the actual output values to the *Monitor* and expected values are routed through the *Generator's* handle. The comparison of actual and expected outputs happens in the *Scoreboard*. The virtual interface and signals in the *Interface* are declared using the constructor. In the class component, the objects of all the components (i.e., driver, monitor, scoreboard, generator, mailboxes) are constructed, and call *run* method of the *Generator*. The environment comprises all the significant components in the test bench. These components are connected in the *Interface*, and the *run* task help to execute the *Monitor* and *Driver* in parallel.

V. EXPERIMENTAL RESULTS

VLSI technology relies heavily on verification since it is used to find bugs in the RTL design at an early stage. The primary purpose of creating a verification environment is to generate stimulus for the design under test (DUT) and check the results to ensure that the DUT functioning is correct. We implemented a verification environment for an AHB-Lite design using System Verilog. AHB-Lite is a simple bus protocol with internal memory devices, external memory interfaces, and high-bandwidth peripherals. Therefore, verification is required at the top level, and the specifications are described at top-level interfaces. For the top-level design, we are using simulation-based verification. To obtain the experimental results we used a simple software simulation and one workstation. We used Synopsys's Verilog Compiler Simulator (VCS) Version R-2020.12 and EDA Playground to run the test cases.

Table I presents the results for the basic signal operation test cases for read and operations. We implemented several test cases to cover different conditions. The read and write transfer with no wait state, burst write operation with multiple cycles, back-to-back multiple read and write operations, read and write with single wait state, and read and write with more than 16 cycles wait state test cases were executed. The outcome of all these test cases were pass. The final test case to read and write with invalid transfer size failed. The test case failed due invalid transfer size. The design showed the error message and transfer was canceled.

The results for the burst operation test cases for read and operations are shown in Table II. We used incrementing and wrapping bursts of 4, 8, and 16 transfer sizes to implement these test cases. The read/write transfer with incrementing and wrapping burst for valid transfer sizes passed the test cases. The read/write transfer with invalid transfer sizes failed

TABLE I: Basic signal operation test cases for read and write operations

Test Case #	Test Case Description	Outcome Status	Remarks
1	Basic write transfer with no wait state with single burst	Pass	Works as expected
2	Basic read transfer with no wait state with single burst	Pass	Works as expected
3	Single burst writes then multiple cycle basic read	Pass	Works as expected
4	Back to back multiple cycles single read and write	Pass	Works as expected
5	Basic read and write then single cycle wait state	Pass	Works as expected
6	Basic read and write then more than 16 cycles wait state	Pass	Works as expected
7	Read and/or write operation with invalid transfer size	Fail	Due to invalid transfer size error is thrown and transfer is canceled

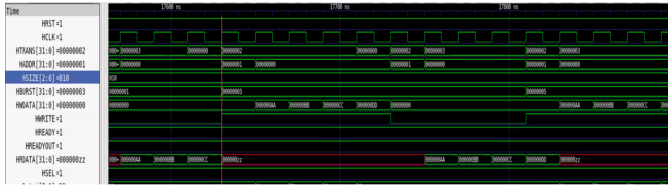


Fig. 7: Increment by 4 burst

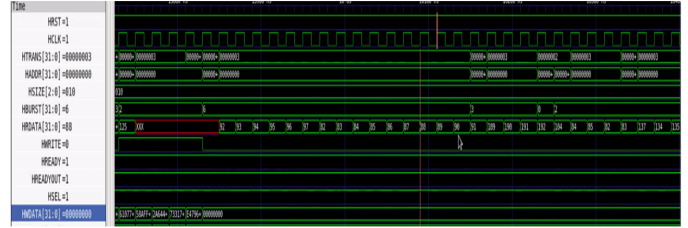


Fig. 9: Wrapping by 16 burst in read mode

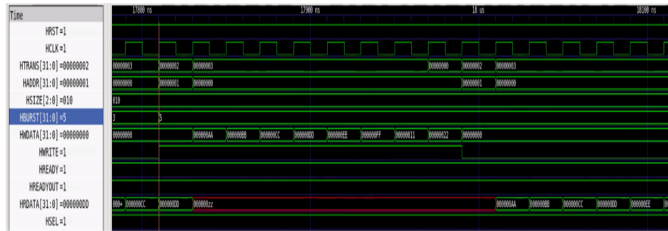


Fig. 8: Increment by 8 burst

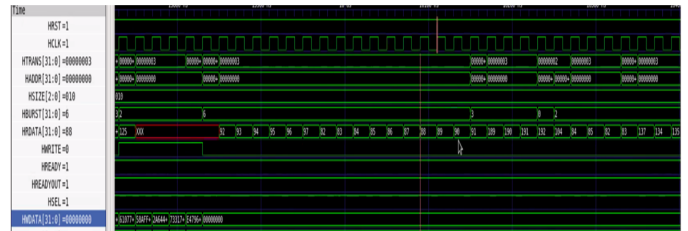


Fig. 10: Different burst modes

the test cases. It failed due to an invalid transfer size. The design throws an error, and the transfer gets canceled. We also performed test cases for back-to-back write transfers, write transfers with incrementing bursts of 4 followed by read transfers and write transfers crossing the 8-bit address boundary. These test cases worked as expected and were passed.

Figure 7 shows the waveform of increment by four bursts. After the transfer type is set to INCR4, the data is written for four cycles without the requirement of any address. As shown in the figure, HBURST=3 indicates INCR4. HBURST signal is a 3-bit signal that indicates the burst type. These signals are generated in the transaction class using constraints and randomization methods. The data (HWDATA) is transmitted to the specified address (HADDR).

Figure 8 shows the simulation waveform of increment by eight bursts. After the transfer type is set to INCR8, the data is written for eight cycles without the requirement of any address. HBURST=5 indicates INCR 8, as shown in the figure. The signals HREADY and HWRITE are set to high. The data (HWDATA) is transferred to the specified address (HADDR).

The simulation waveform of wrapping by 16 bursts in read mode is shown in Figure 9. After the transfer type is set to WRAP16, the data is written for 16 cycles without the requirement of any address with wrapping. As shown in the

figure, HBURST=6 indicates WRAP 16. The data (HRDATA) is transmitted to the specified address (HADDR).

Figure 11 indicates different types of burst modes. The figure shows that different HBURST values are pushed as stimuli and checked for functionality. HBURST is a 3-bit signal. When the value of HBURST is 0, one burst mode is set. For HBURST=1 mode is incremental, HBURST=2 mode is wrap by four, HBURST=3 mode is increment by four, HBURST=4 mode is wrap by eight, HBURST=5 mode is increment by eight, HBURST=6 mode is wrap by 16, and HBURST=7 mode is increment by 16.

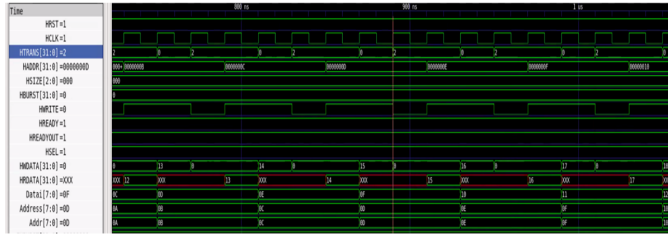
The read and write transfers in alternative cycles are shown in Figure 11. The figure shows that the HWRITE signal continuously toggles for every cycle. The HWRITE=1 indicates writing to the slave, and HWRITE=0 indicates reading from the slave. The signals HREADY, HREADYOUT, and HSEL are asserted to perform read and write operations.

VI. CONCLUSION

We designed AHB in this work to support a single master and two slaves. The System Verilog testbench is used to validate the AHB-Lite protocol. We encountered several challenges while developing the testbench environment. The importance of pre-silicon coverage is studied and understood. The checker comes in handy for detecting internal bugs during

TABLE II: Burst operation test cases for read and write operation

Test Case #	Test Case Description	Outcome Status	Remarks
1	Read/Write transfer with incrementing burst of undefined length for valid transfer sizes	Pass	Works as expected
2	Read/Write transfer with incrementing burst of undefined length for invalid transfer sizes	Pass	Due to invalid transfer size, design throws error and transfer cancelled
3	Read/Write transfer with incrementing burst of 4, 8, 16 for valid transfer sizes	Fail	Works as expected
4	Read/Write transfer with incrementing burst of 4, 8, 16 for invalid transfer sizes	Fail	Due to invalid transfer size, design throws error and transfer cancelled
5	Read/Write transfer with wrapping burst of 4, 8, 16 for valid transfer sizes	Pass	Works as expected
6	Read/Write transfer with wrapping burst of 4, 8, 16 for invalid transfer sizes	Pass	Works as expected
7	Write transfer with incrementing burst of 4, then Read transfer with incrementing burst of undefined length	Pass	Works as expected
8	Write transfer with incrementing burst of 4, then Read transfer with wrapping burst of length 4	Pass	Works as expected
9	Write transfer with incrementing burst of 4, then by incrementing the length of 4, then Read transfer with incrementing burst of length 8 starts from the initial write address	Pass	Works as expected
10	Back-to-back Write transfers with different burst size	Pass	Unable to get the expected output since its data bus is only limited to 1 byte. As the implementation of tests is in size (more than 1 byte), the design fails. Hence test passed.
11	Back-to-back four write transfers of burst size 1, 2, 3, and 4	Pass	Works as expected
12	Write transfer with incrementing burst crossing an 8-bit address boundary	Pass	As the slave address burst width is 8-bit and the the design doesn't allow slaves to switch in the middle of burst, and design breaks.

**Fig. 11:** Read and write transfers

simulation. The requirement to create test cases taught us how critical it is for a validation engineer to understand critical situations in a design. We understand risks and dependencies in a broader sense as a team during the development process. The test bench is built, and the design is validated with possible test cases. The Synopsys VCS tool is an EDA tool used to simulate and verify the design, as well as to obtain a coverage report that indicates functional correctness. The System Verilog report summary ensures that the design is functionally correct.

REFERENCES

- [1] A. Sharma and Z. Khan Ali, "Construct high-speed sdr memory controller using multiple fifo's for ahb memory slave interface," 2013.
- [2] S. L. Reddy and A. K. Kumari, "Architecture of an ahb compliant sdr memory controller," 2013.
- [3] M. M. Kareemullah Shaik and M. Zahirullah, "A reduced latency architecture for obtaining high system performance," 2012.
- [4] A. C. Vishwakarma, Varsha and A. Sahu, "Implementation of amba ahb protocol for high capacity memory management using vhd1," 2012.
- [5] M. B. R. Srinivas and S. Musala, "Verification of ahb_lite protocol for waited transfer responses using re-usable verification methodology," in *2016 International Conference on Inventive Computation Technologies (ICICT)*, vol. 3, 2016, pp. 1–3.

- [6] M. Dubois, Y. Savaria, and G. Bois, "A generic ahb bus for implementing high-speed locally synchronous islands," in *Proceedings. IEEE SoutheastCon, 2005.*, 2005, pp. 11–16.
- [7] L. Deeksha and B. R. Shivakumar, "Effective design and implementation of amba ahb bus protocol using verilog," in *2019 International Conference on Intelligent Sustainable Systems (ICISS)*, 2019, pp. 1–5.
- [8] J. Romero, N. Cuevas, and E. Roa, "Energy efficient peripheral and system buses for low-area and low-power soc applications," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 67, no. 5, pp. 866–870, 2020.